

SO2 - Etapa 5

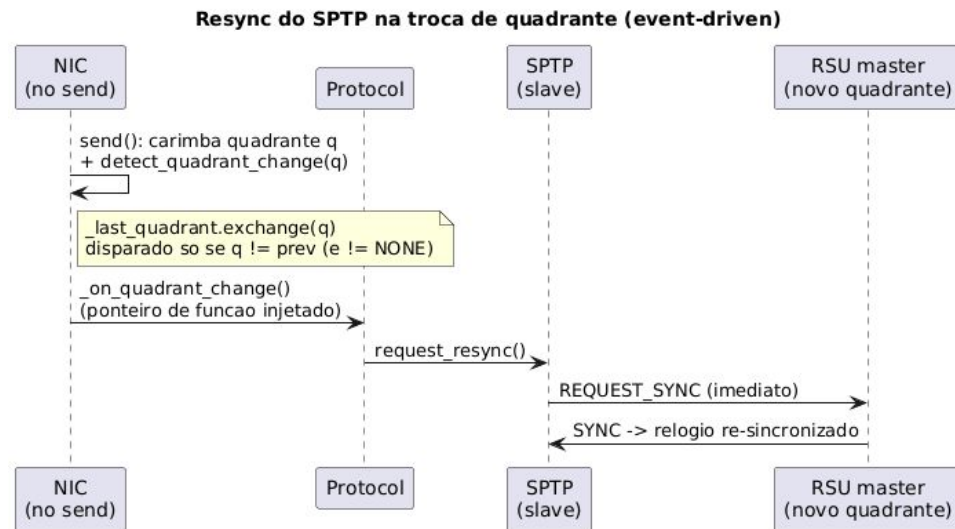
Time-Triggered Publish-Subscribe Messages
Grupo M3

Integrantes

Arthur Erpen, Caetano Peruzzo e Gabriel Herrera

● Correção Etapa 4

- Implementamos a ressincronização do PTP nas trocas de quadrante.
- NIC guarda o último quadrante em um atômico `_last_quadrant`. No caminho do send ela já carimba o quadrante no frame e, ali mesmo, chama `detect_quadrant_change(q)`, que é um exchange, que dispara apenas na transição de quadrante.
- NIC não conhece o SPTP, ela chama um handler “`on_quadrant_change`” que o protocol injetou via `setup_quadrant_change_handler`
- `Protocol::on_quadrant_change()` chama `SPTP::request_resync()`, que, por sua vez, envia um `REQUEST_SYNC` imediato (apenas no slave, master ignora).



● O Problema

I — Interesse

{ origin, timestamp, type, period }



origin



timestamp



type
(código TEDS)



period

R — Resposta

{ origin, timestamp, type, value }



origin



timestamp



type
(código TEDS)



value

Regras principais



Tudo é **broadcast**.



O tipo é identificado por **código TEDS** (IEEE 1451).



O produtor responde **periodicamente** enquanto houver interesse.



Sem eventos. **Sem** publicação explícita.

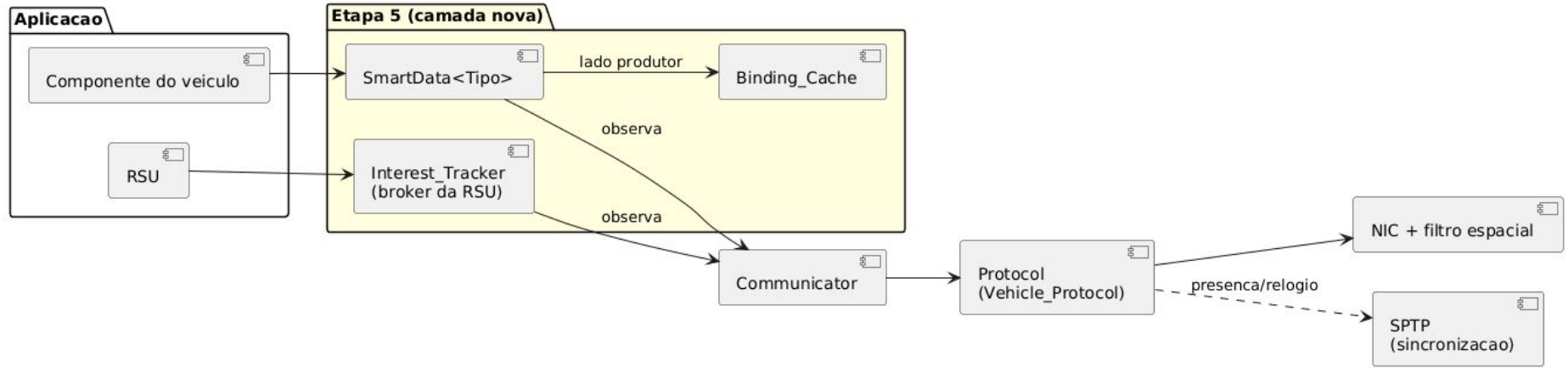


Sincronização temporal presumida (**SPTP** em execução de forma transparente).



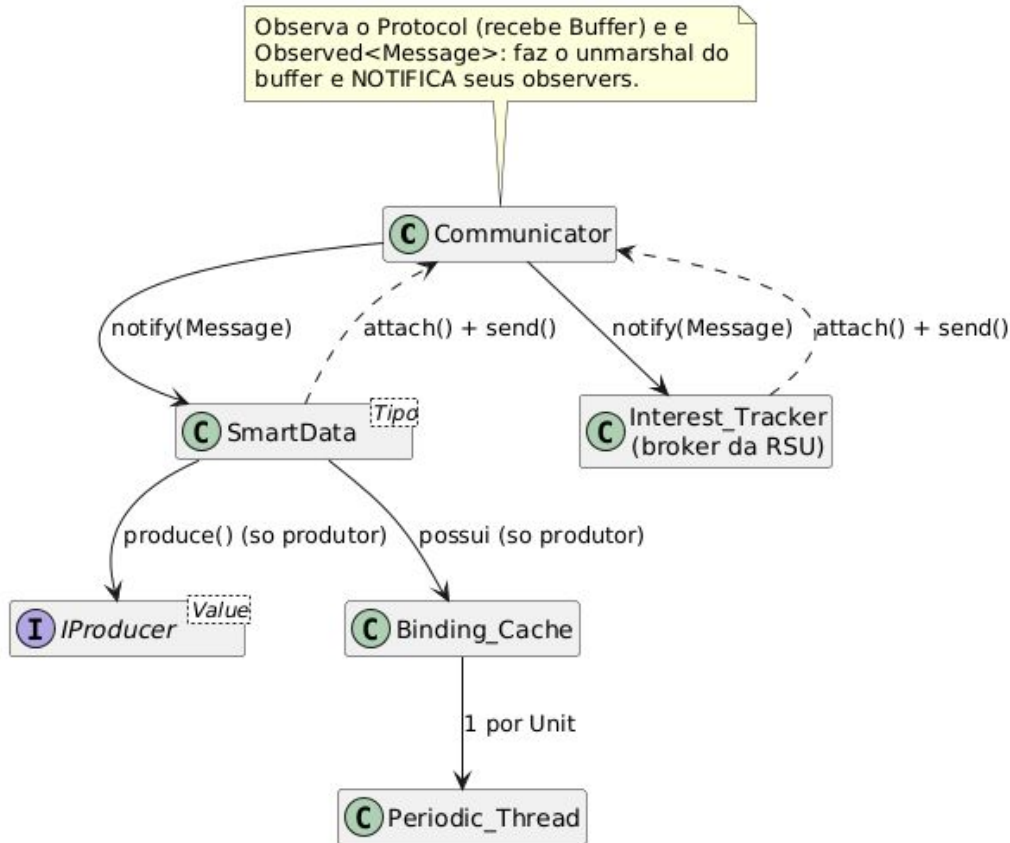
Quem quer um dado anuncia interesse; quem produz aquele tipo responde no período pedido — e fica respondendo até não haver mais interessados.

- Onde a Etapa 5 Entra na Pilha



- Adicionou-se uma camada que fala interesse/resposta usando o Communicator.
- RSU ganhou um broker que vive na mesma camada.

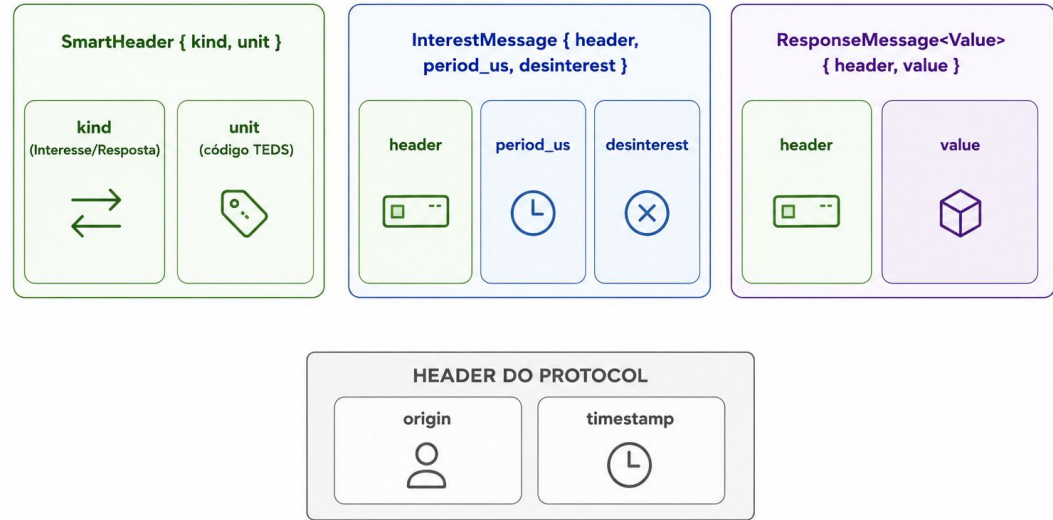
• Como o dado chega: a cadeia de observers



- O Communicator observa o protocolo (recebe buffers) e é observável por Message (faz unmarshall do buffer e notifica).
- O SmartData observa o Communicator, no `update()` filtra por kind (Interesse/Resposta) e por unit (TEDS).
- Padrão Observer/Observed desenvolvido nas etapas anteriores foi reusado: o channel empurra a mensagem pro SmartData, a aplicação apenas lê o resultado.

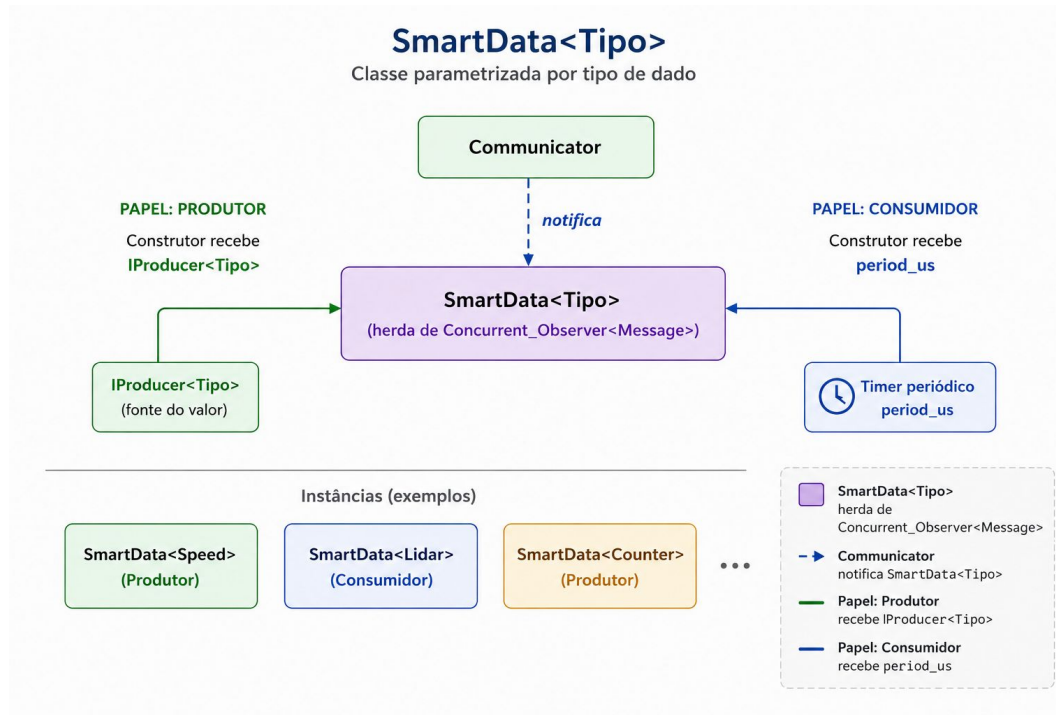
● Modelo de Mensagens

- **SmartHeader{kind, unit} →**
‘kind’ é Interesse/Resposta,
unit é o código TEDS.
- **InterestMessage{header, period_us, desinterest}**
- **ResponseMessage<Value>{header, value}**
- **origin/timestamp** ainda viajam no header do Protocol



● SmartData: classe central

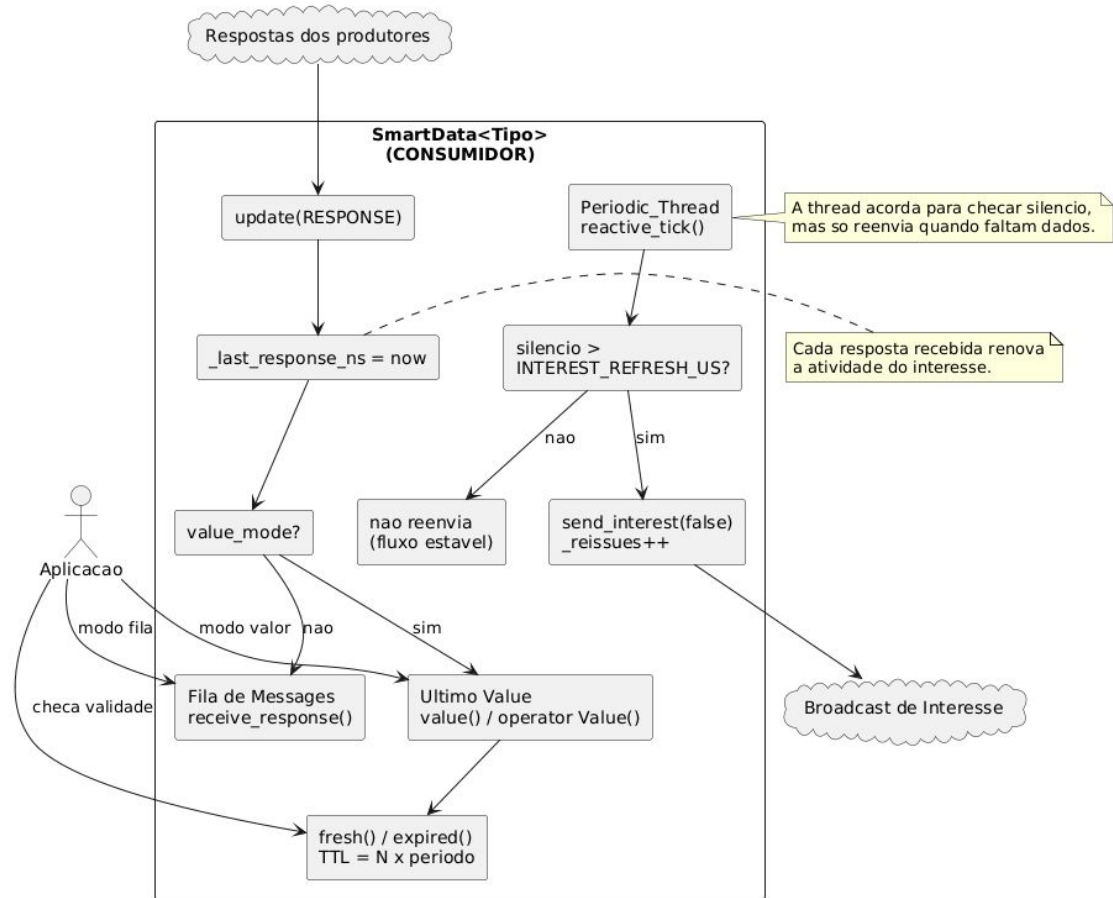
- `template<typename Tipo>`
`class SmartData` → uma instância por tipo de dado.
- Dois papéis pelo construtor: produtor (recebe um **IProducer**, a fonte do valor) e consumidor (recebe o período).
- Herda de **Concurrent_Observer<Message>** → observa o Communicator.
- Em síntese, mesma classe, mas dois papéis. O tipo do valor é parâmetro de template (Speed, Lidar, Counter...).



- **Lado produtor: Binding_Cache + thread periódica**
 - Ao receber interesse: `Binding_Cache.on_interest(unit, address, period)`. Guarda binding **por (Unit, Endereço)**.
 - Uma `Periodic_Thread` por Unit, no período mais curto entre os interessados.
 - Soft-state: cada binding tem validade. Uma tarefa de limpeza periódica remove os expirados. O desinteresse remove só o endereço daquele agente.

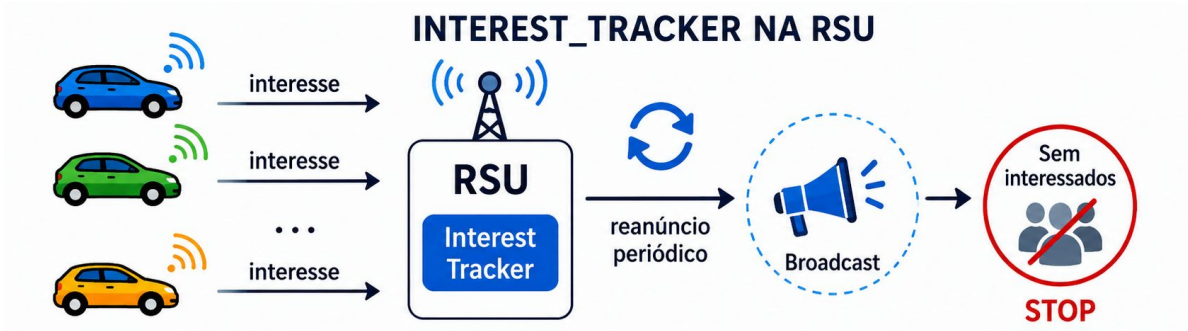
● Lado consumidor: refresh reativo + modo valor

- Refresh reativo → só reenvia o interesse após silêncio de dados (não fica em polling) - variável `_last_response_ns`.
- Dois modos de leitura: fila (`receive_response()`) ou modo valor (`value()` / `operator Value() + fresh()` / `expired()`), nesse caso o dado é tratado como variável viva.

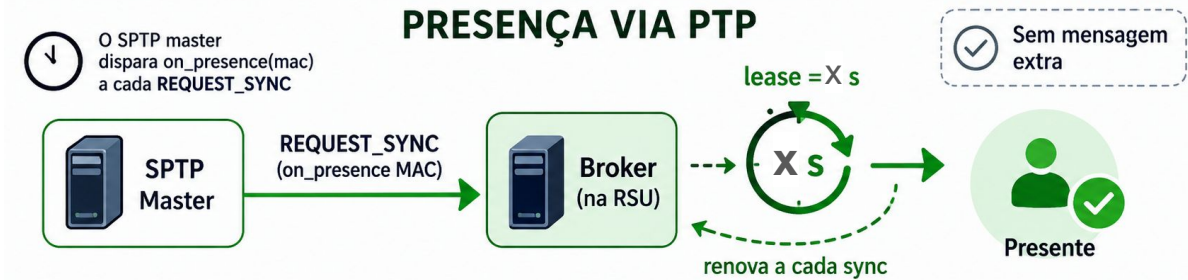


● RSU como broker (rastreamento passivo)

- Interest_Tracker na RSU: ouve os interesses, reanuncia periodicamente (cobre perdas e late joiners) e manda parar quando uma Unit fica sem ninguém.

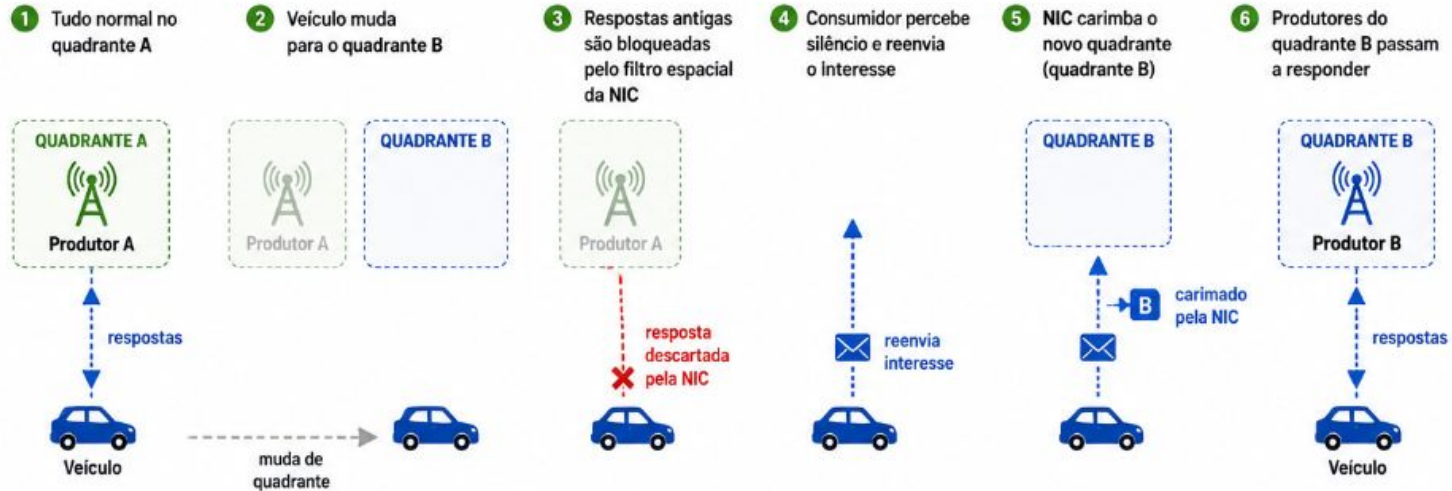


- Presença via PTP: o SPTP master dispara on_presence(mac) a cada REQUEST_SYNC. O broker mantém um lease (4s). Sem mensagem extra.



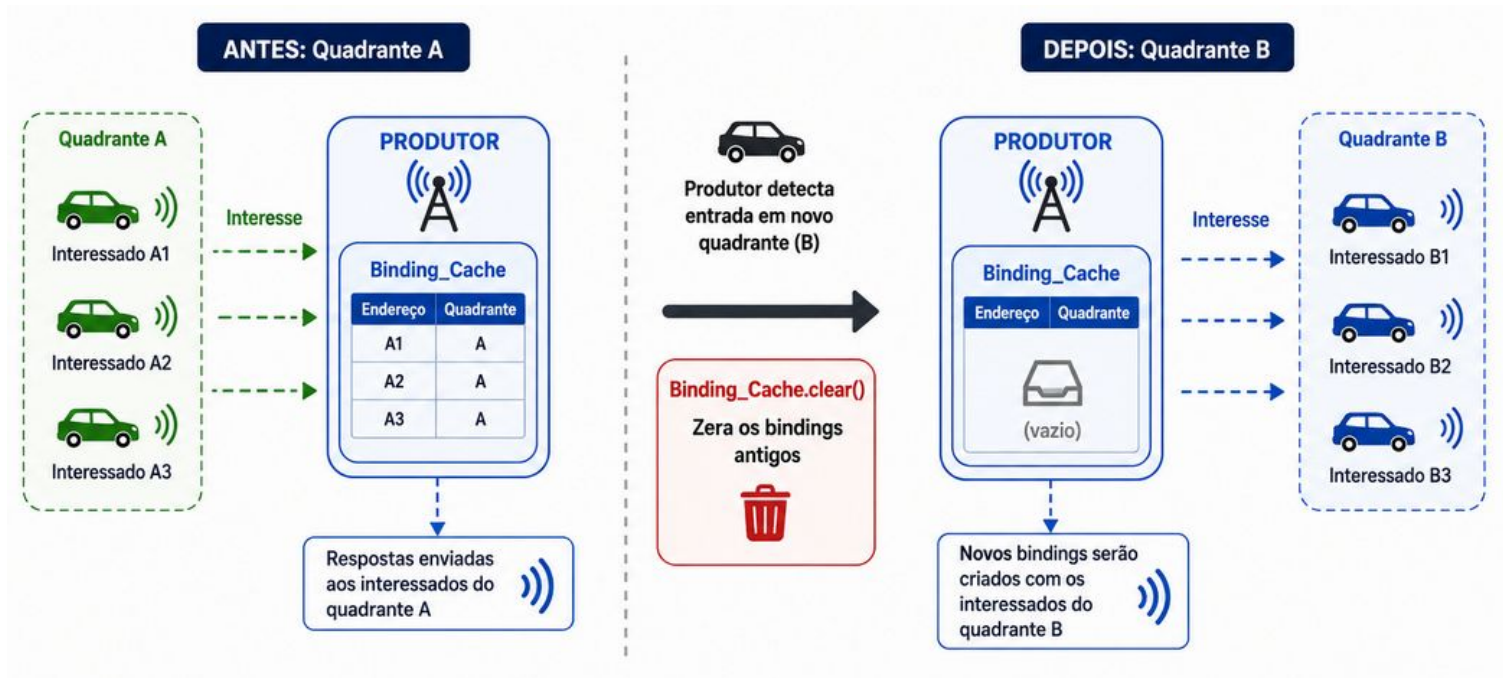
● Mobilidade implícita (lado consumidor)

- O cliente não conhece quadrante. O filtro espacial da NIC corta frames de outro quadrante.
- Caso o veículo troque de quadrante e fique em silêncio por um período de tempo, o reenvio reativo ocorre, onde outra mensagem de interesse é enviada para o novo quadrante.



● Mobilidade: lado produtor

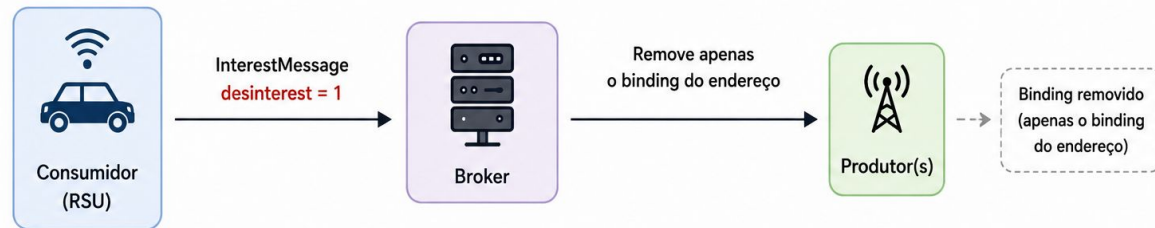
- Quando o produtor detecta entrada em novo quadrante, `Binding_Cache.clear()` zera os bindings antigos (os interessados do quadrante anterior já não o alcançam).



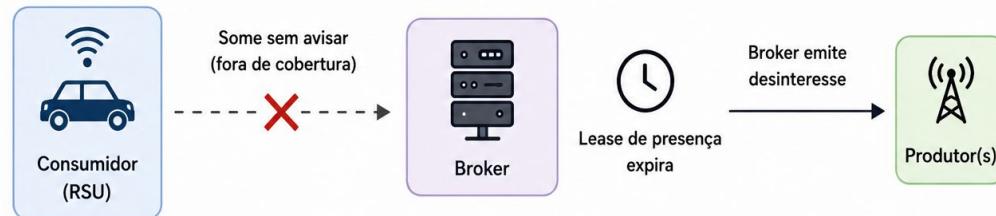
● Desinteresse: dois caminhos

- Colocar bit explícito na InterestMessage
desinterest = 1 → produtor remove só o binding daquele endereço.
- Por presença (RSU) → consumidor some sem avisar → lease de presença expira → broker emite o desinteresse aos produtores.

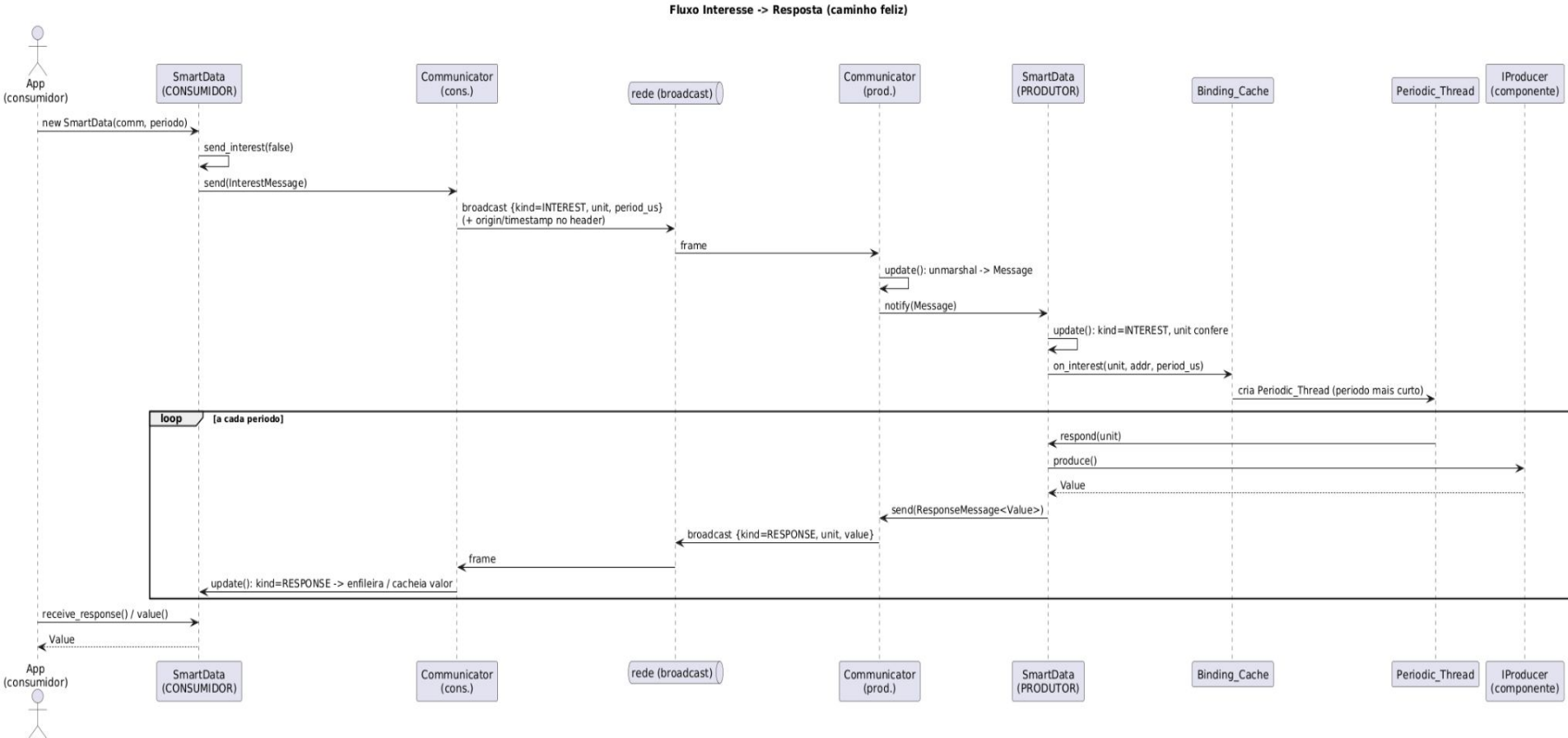
1) Desinteresse explícito (bit = 1)



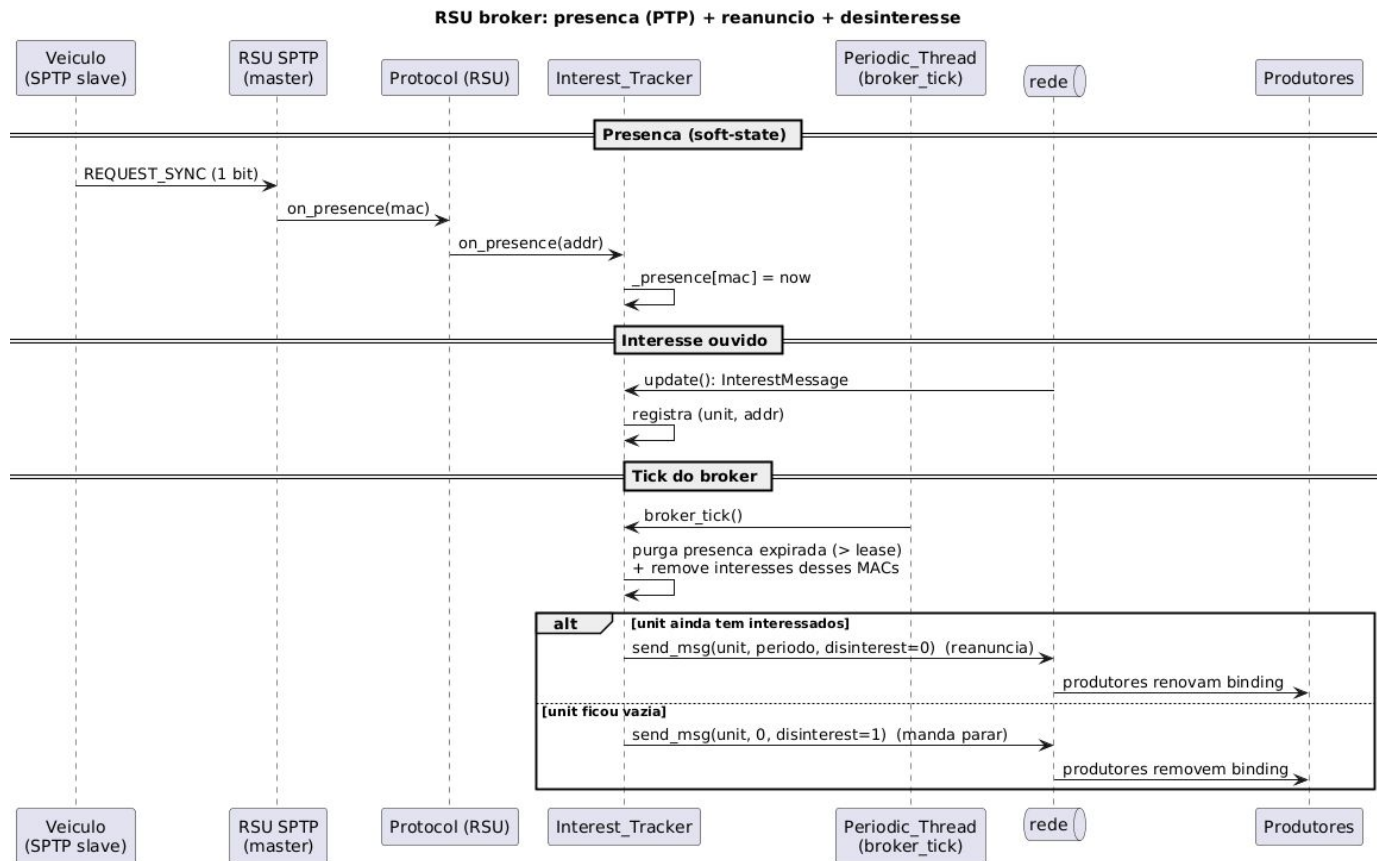
2) Por presença (RSU)



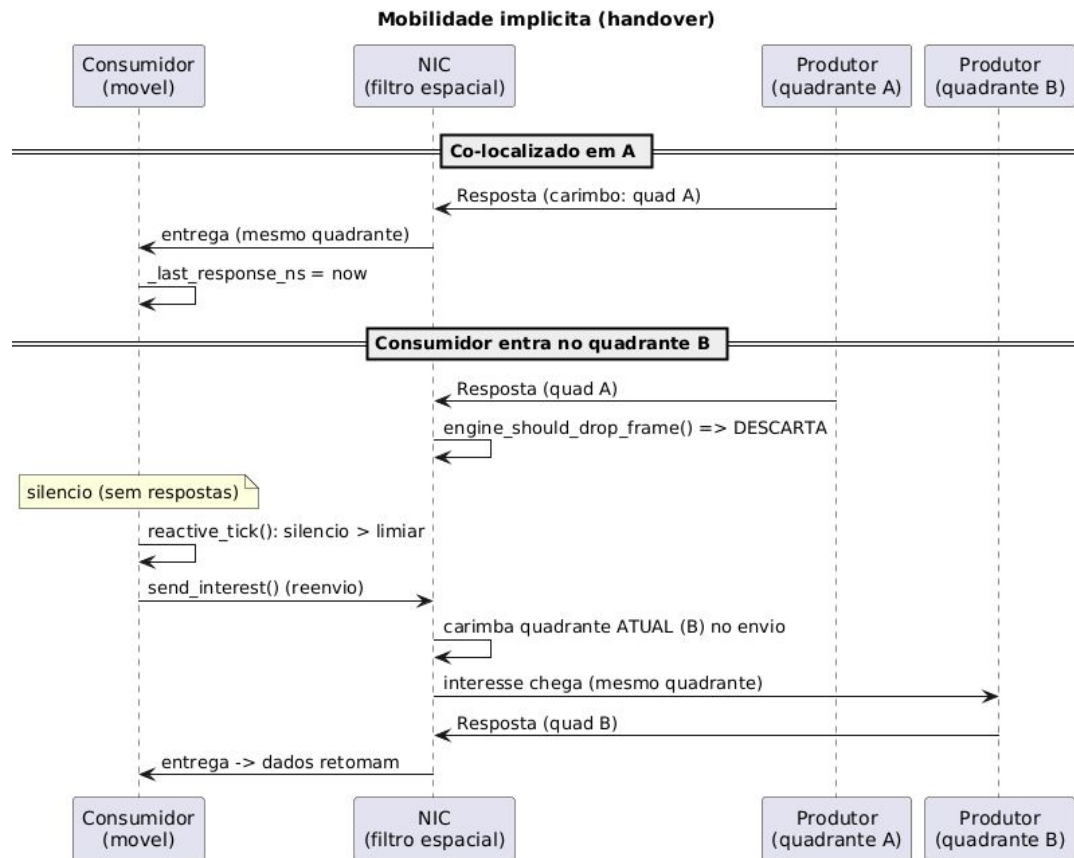
● Fluxo: Interesse → Resposta



● Fluxo: RSU broker (presença + reanúncio + desinteresse)



● Fluxo: troca de quadrante



● Overview

Produtor: componente que pode gerar e transmitir dados de uma unidade específica. Ele gerencia uma lista dinâmica de Interessados, cuja taxa de envio é ditada pela maior frequência registrada entre as solicitações ativas. o ciclo de vida dessa lista possui algumas regras:

- Sempre que o produtor recebe uma mensagem de interesse ele pode adicionar uma nova entrada a lista de interesse, ou atualizar as informações de uma entrada já existente.
- Se uma mensagem de desinteresse é recebida e essa colide com alguma entrada de interesse da lista essa é removida.
- Entradas antigas expiram e são removidas se não forem renovadas pelo Interessado/RSU.
- A lista de interesses é completamente resetada sempre que o veículo muda de quadrante.

- **Overview**

Consumidor/Interessado: o componente que manifesta a necessidade de receber um tipo de dado específico. Seu comportamento baseia-se em:

- **Declaração Inicial:** Declara seu interesse uma única vez assim que é inicializado no sistema.
- **Redeclaração Preventiva (Keep-alive):** Caso passe um período prolongado sem receber nenhuma atualização do dado esperado, ele retransmite a mensagem de interesse para reativar o fluxo (troca de quadrantes implicitamente).

- **Overview**

RSU: atua como o gerenciador de tráfego de dados e presença do quadrante através de duas frentes:

- **Retransmissão de Interesses:** Escuta passivamente as mensagens de interesse geradas no quadrante e as retransmite periodicamente para garantir o alcance entre as VMs.
- **Monitoramento e Expulsão (PTP):** Monitora a presença dos veículos localmente via troca de mensagens PTP. Se um veículo cessar o envio dessas mensagens de sincronização, a RSU assume que ele deixou o quadrante. Como consequência, ela remove os interesses desse veículo de sua lista de retransmissão e propaga mensagens de **desinteresse** para cessar os envios dos produtores locais.

● Testes

- **fanin:** 1 VM consumidora, 5 VMs produtoras. Testa o caso base de “1 manda interesse, vários respondem”.
- **fanout:** 1 VM produtora, 4 VMs consumidoras, sendo que os consumidores pedem o dado em frequências diferentes. Testa o comportamento do produtor de produzir na frequência mais alta requisitada.
- **types:** 4 VMs consumidoras, cada uma com seu tipo de dado, 6 VMs produtoras, que englobam pelo menos os 4 tipos. Testa a multiplexação por tipo de dado
- **period:** 1 VM consumidora, 20 VMs produtoras, a consumidora mede o intervalo de respostas de cada uma das produtoras. Testa se o intervalo de resposta se mantém dentro de um limite aceitável, mesmo com muitas produtoras.
- **lifecycle:** 2 VMs consumidoras, 4 VMs produtoras, todas do mesmo dado, eventualmente uma das consumidoras “sai”. Testa que a saída (e desinteresse) de uma VM que quer um tipo de dado não apaga os interesses de outras VMs que querem o mesmo dado

● Testes

- **value:** 1 VM produtora, 1 VM consumidora. Testa o caso base Interesse/Publicação, mas a consumidora lê o dado como variável viva, usando *fresh* para verificar a validade do dado. Prova o caminho de leitura estilo SmartData/EPOS.
- **presence:** 1 VM produtora, 1 VM consumidora, eventualmente a consumidora sai abruptamente, sem enviar desinteresse. Testa o caso no qual a RSU percebe que uma consumidora/interessada não está mais presente, e envia desinteresse para os interesses registrados daquela VM.
- **rsu-repeat:** 1 VM produtora, 1 VM consumidora, a consumidora “entra” depois da mensagem de interesse. Prova que, devido a retransmissão de interesses pela RSU, produtores “atrasados” ainda ficam sabendo de interesses vigentes na região.
- **mobility:** 3 VMs produtoras, 2 VM consumidoras, conforme as VMs vão trocando de quadrantes os produtores e consumidores vão se esquecendo e se reencontrando. testa o re-anúncio de interesse do consumidor quando não recebe dados.

● Testes

- **scale:** 20 VMs, cada uma tem 5 componentes, dos quais 4 são produtores, 1 é consumidor. Cada um dos produtores é de um tipo (speed, lidar, radar, counter), e existe pelo menos um consumidor de cada tipo. Testa o funcionamento do sistema em um cenário “real”

Observação 1: somente os testes de **Scale** e **Mobility** consideram o cenário total com 4 quadrantes e 4 RSU fixas, o restante dos casos de testes consideram somente um quadrante com 1 RSU

Observação 2: nosso comando “make” roda toda suíte de testes da etapa 5 (10 testes), que demoraram, em média, 8 minutos para serem executados, mas cada um dos testes pode ser rodado individualmente utilizando o comando “make test-interest-<nome_do_teste>”.

Resultados: `measure-rtt`

Cenario: latencia inter-VM via raw socket

Amostras: 20 RTTs

Metrica	Resultado
RTT medio	11.1669 ms
RTT minimo	6.3350 ms
RTT maximo	33.0190 ms
Latencia estimada de ida	5.5835 ms
Latencia minima estimada de ida	3.1675 ms
Latencia maxima estimada de ida	16.5095 ms

Resultados: `measure-rtt-intra`

Cenário: latencia intra-VM via memoria compartilhada

Amostras: 5000 RTTs

Metrica	Resultado
RTT medio	1.8939 ms
RTT minimo	1.4580 ms
RTT maximo	27.3130 ms
Latencia estimada de ida	0.9469 ms
Latencia minima estimada de ida	0.7290 ms
Latencia maxima estimada de ida	13.6565 ms

OBRIGADO!